Negative to Positive Converter — Architecture & Design Reference

Written March 2026. Covers the app as built, not the original spec.

Purpose

A macOS App Store app that converts scanned film negatives (color and black & white) to positive images with minimal user interaction. The app also has an Enhance mode for improving existing positive scans. Output files are written to a user-selected output folder, named with a `_p` (convert) or `_e` (enhance) suffix.

Source Files

File	Responsibility
`ContentView.swift`	All UI, routing, file I/O, App entry point
`ConversionLogicColor.swift`	Color negative pipeline + enhance
`ConversionLogicBW.swift`	B&W negative pipeline + enhance
`ColorOrBW.swift`	Image type classifier
`OrangeChallenge.swift`	Early experimental orange removal (not used in main pipeline)

Application Modes

App Mode (top-level):

- **Convert Negatives** — inverts a negative scan to a positive
- **Enhance Images** — improves an existing positive scan

Process Mode (affects both app modes):

- **Automatic** — straight conversion, no post-processing
- **Balanced** — applies brightness and contrast correction after conversion
- **Use Reference Image** — matches tonal and color characteristics to a reference positive

---

Image Classification (`ColorOrBW.swift`)

Before processing, every image is classified as color or B&W by `ImageTypeAnalyzer`. It scales the image to 256px, samples every 10th pixel, and counts pixels where `max(R,G,B) - min(R,G,B) > 15`. If fewer than 5% of sampled pixels show color variation, the image is classified as B&W.

Routing Logic (`ContentView.swift` → `processImages`)

The routing matrix determines which processor and which reference path is used:

Source type	Reference	Route
Color	None	`colorProcessor.convertImageColor`
Color	Color ref	`colorProcessor.convertImageColor` (applyReference)
Color	B&W ref	`colorProcessor.convertImageColor` (desaturate + histogramMatchLuminance)
B&W	None	`bwProcessor.convertImageBW`
B&W	Color ref	`colorProcessor.convertImageColor` (applyLabABTransfer, sourceIsGrayscale=true)
B&W	B&W ref	`colorProcessor.convertImageColor` (histogramMatchLuminance, transferLuminance=true)

The same matrix applies for Enhance mode, substituting `enhancePositiveColor` and `enhancePositiveBW`.

`referenceIsBW` is determined by running the classifier on the reference image before the batch loop. This flag drives the `transferLuminance` parameter.

Color Pipeline (`ConversionLogicColor.swift`)

Convert Negatives path:

1. **Load** — JPEG/PNG/TIFF via `CIImage(contentsOf:)`, RAW via `CGImageSourceCreateThumbnailAtIndex` at max 4000px
2. **Orange detection** — sample brightest 5% of pixels (by luminance) from inner 80% of image, average their RGB → this is the film base color

3. **Orange removal** — per-channel gain: `gain = 1/orange_channel`, normalised so the largest gain = 1.0, applied as a multiply kernel
4. **Tonal stretch** — per-channel, 0.5th–99.5th percentile endpoints mapped to [0,1]. Rejects dust/scratch outliers
5. **Invert** — `1.0 - value` per channel
6. **Reference matching** (optional, see below)
7. **Balanced corrections** (optional, see below)
8. **Save**

Enhance Positives path:

1. **Load**
2. **Tonal stretch** (same percentile approach)
3. **Reference matching** (optional)
4. **Balanced corrections** (optional)
5. **Save**

B&W Pipeline (`ConversionLogicBW.swift`)

Convert Negatives path:

1. **Load**
2. **Grayscale conversion** — Rec.601 luminance weights (0.299R + 0.587G + 0.114B)
3. **Endpoint sampling** — 0.5th and 99.5th percentile of inner 80%
4. **Stretch** — linear map from [darkPoint, brightPoint] to [0,1]
5. **Invert**
6. **Balanced corrections** (optional)
7. **Save as true grayscale JPEG** — written via a grayscale CGContext, not RGB

Enhance Positives path:

1. **Load**
2. **Grayscale conversion**
3. **Endpoint sampling + stretch**
4. **Balanced corrections** (optional)
5. **Save**

Reference Matching — Three Paths

Path 1: Color source + Color reference (`applyReference`)

Used when both source and reference are color images. Works in RGB:

- Sample middle 80% of tonal range from both images (excludes blown highlights and pure blacks)
- Per-channel std deviation ratio → contrast gain, capped at 1.5, pivoted around 0.5
- Per-channel mean ratio → color temperature gain, capped at 1.0 (channels can only be pulled down)
- Apply combined gain, then stretch output to full [0,1] per channel
- Result: reference controls contrast shape and color balance only; source brightness is self-determined

Path 2: Color source + B&W reference (`desaturate` + `histogramMatchLuminance`)

- Step 1: Desaturate the positive using Rec.709 luminance weights
- Step 2: Histogram-match the luminance to the reference using stable-zone std deviation ratio, capped, stretched
- Result: B&W output with reference's tonal character

Path 3: B&W source + Color reference (`applyLabABTransfer`)

Used when a greyscale source gets a color reference to introduce toning (e.g. blue reference → blue-toned print, warm paper reference → warm-toned print).

- Analyse both source and reference in Lab space using a 3×3 regional grid
- Transfer **a and b channels only** (color/chroma) from reference to source — L channel untouched
- This directly transplants the hue and chroma of the reference while keeping source luminance
- Highlight/shadow compression applied via soft knee rolloff in the GLSL kernel
- Result: color-toned greyscale image

Balanced Mode (`normalizeMidtonesColor`, `balanceContrastColor`, and BW equivalents)

Applied after conversion/reference matching when Process Mode = Balanced. Both brightness and contrast corrections always run (they were previously separate checkboxes, removed for simplicity — code separation preserved).

Brightness correction

- Measures mean Lab L* of the image (color) or median luminance (B&W)
- Computes distance from target (currently L*=78 for color, 0.78 for B&W — intentionally above middle grey)
- Applies only a fraction of that distance: `maxBrightnessCorrection` (currently 0.65)
- Dark images get brighter, bright images get slightly darker — bidirectional
- Implemented as gamma on L channel only in Lab space → color balance unaffected
- Naturally self-limiting: highlights near 100 L* barely move

Contrast correction

- Measures L* std deviation
- **Boost only — never reduces contrast**
- Skipped if std is within `contrastDeadzone` (85%) of `targetLStd`
- S-curve sigmoid on L channel only in Lab space → color balance unaffected
- Steepness k proportional to how flat the image is, capped at `maxContrastK`

Tuning constants (top of each logic file, easy to find):

`ConversionLogicColor.swift`:

```swift
private let targetLMean:             Double = 78.0   // brightness target (Lab L*)
private let maxBrightnessCorrection: Double = 0.65   // correction strength 0.0-1.0
private let targetLStd:              Double = 22.0   // contrast target (L* std dev)
private let maxContrastK:            Float  = 6.0    // sigmoid steepness cap
private let contrastDeadzone:        Double = 0.85   // dead zone fraction
```

`ConversionLogicBW.swift`:

```swift
private let bwTargetMedian:            Double = 0.78  // brightness target (0-1)
private let bwMaxBrightnessCorrection: Double = 0.65  // correction strength
private let bwTargetStd:               Double = 0.22  // contrast target
private let bwMaxContrastK:            Float  = 6.0   // sigmoid steepness cap
private let bwContrastDeadzone:        Double = 0.85  // dead zone fraction
```

Output Folder & Sandbox

The app targets the Mac App Store with App Sandbox enabled. The entitlements are:

- `com.apple.security.app-sandbox = true`
- `com.apple.security.files.user-selected.read-write = true`

The sandbox only grants access to files the user explicitly selects — not their parent directories. Writing a new output file next to the source is therefore blocked. The solution:

- User selects an output folder once via `NSOpenPanel`
- A **security-scoped bookmark** is saved to `UserDefaults` (`outputDirectoryBookmark` key)
- On every subsequent launch, the bookmark is resolved and `startAccessingSecurityScopedResource()` is called — sandbox access is restored automatically
- Survives app updates, reboots, and Xcode clean builds
- If no output folder is set and the user presses Convert, a clear message is shown and conversion is blocked

Drag-and-drop uses `NSItemProvider.loadItem(forTypeIdentifier: "public.file-url")` — not `loadFileRepresentation` — to preserve the original security-scoped URL rather than receiving a temp copy.

UI Structure (`ContentView.swift`)

Three-panel horizontal layout with an action bar below:

Input panel — mode selection (Convert/Enhance), drag-and-drop zone, file list, Add/Clear buttons

Process panel — Automatic / Balanced / Use Reference Image radio buttons, reference image drop zone and picker

Output panel — preview of last converted image, JPEG/TIFF format selector, output folder path and Select button

Action bar — Convert/Enhance button with progress indicator

Single `.fileImporter` handles all three picker cases (negatives, reference, output folder) via `PickerMode` enum: `.negatives`, `.reference`, `.outputFolder`.

Key Technical Decisions & Their Reasons

Decision	Reason
Lab color space for brightness/contrast	Correcting in RGB shifts color balance; Lab L is luminance-only
Per-channel tonal stretch before invert	Normalises each channel independently, removing color cast from uneven exposure
Orange removal by sampling brightest 5%	Film base is always the brightest area of a negative; this finds it robustly without assumptions about film stock
Percentile endpoints (0.5/99.5) not min/max	Dust, scratches, and light leaks would otherwise clip the entire tonal range
Contrast boost only, never reduce	Compression discards image information; expansion reveals what's already there
Security-scoped bookmarks for output folder	The only App Store-compliant way to write to arbitrary user-chosen directories persistently
Single fileImporter for all pickers	SwiftUI only supports one .fileImporter per view; multiple modifiers silently break each other
`loadItem` not `loadFileRepresentation` for drag-and-drop	`loadFileRepresentation` copies the file to a temp location, losing the sandbox-scoped URL

What `OrangeChallenge.swift` Is

An earlier experimental approach to orange mask removal that sampled the film base by excluding the center of the frame (assuming subject matter in the center, pure film base at edges). It is **not used** in the current pipeline — orange detection was simplified and moved directly into `ConversionLogicColor.swift`. The file is retained for reference.